

develop an API from scratch.

high	
Control over our data	Observability and control are not given away. We can decide if and when we want to migrate to any other solution. We can decide on where our data is stored, level of redundancy air gap backups, ect.
medium	
Future proof	When there is a better or more desirable solution available in the future we are not restricted in any way to migrate.
high	
No product lock-in	We are not locked in to one product we can not leave in the future.
medium	
Debugging capability	Compared to any SaaS offering we do not rely on a vendor to be willing and able to debug our problems in a timely manner.
high	
Good integration with Cells	Compared to other self-hosted solutions, the database will run in the same k8s cluster as the rest of the workloads. This removes the need to integrate external components as well as multiple failure vectors.
medium	
Near Zero Downtime Upgrade	We can adapt Gitlab's (db-migration/pg-upgrade-logical Automation to achieve near-zero downtime for PostgreSQL MVU over k8s
high / blocker	

Cons / Risks	Description
Priority	
-----	-----
-----	-----
Know how	k8s is widely used among our infrastructure, but not within Database Reliability. We need to build up knowledge in the team.
medium	
Missing Features	Missing features need to be implemented by us, e.g. not supported extension.
medium	

Things to validate

- Compare Candidates (4-5 weeks)
- CYBERTEC PostgreSQL Operator (zalando with commercial support)
- CloudNativePG

Amazon RDS PostgreSQL

Amazon Relational Database Services PostgreSQL is AWS's managed database service offering fully compatible with PostgreSQL community version. In fact, Amazon only packs and deploys the PostgreSQL community binaries into the RDS instance underlying infrastructure. GitLab currently recognizes Amazon RDS PostgreSQL as a supported PostgreSQL implementation.

Pro	Description
Priority / Importance	
-----	-----
-----	-----
-----	-----
-----	-----
-----	-----

DBaaS	Minimal operation overhead, in theory. We only need to define what DB we need and get it provided as a Service.
high	
Outsourced	We don't need to maintain infrastructure beyond our hooks, interfaces, monitoring, alerting and forecast.
high	
Ops. Extensions	The essential extensions necessary to run GitLab.com are available.
high	
API	An API for all offered features is already provided.
high	
Gitlab Dedicated Compatibility	Amazon RDS PostgreSQL is already used by our Gitlab Dedicated architecture, therefore is already integrated into Gitlab Dedicated-tooling
?	
Less Risk of Vendor lock-in	Currently, we are not fully locked into the RDS infrastructure. Migration is possible via PostgreSQL native logical replication, Export to S3, or even Amazon DMS, which supports migrating RDS databases to "on-premises" medium
Commitment to PostgreSQL development	Amazon currently employs 9 committers and major contributors to PostgreSQL source code. In contrast, Microsoft employs 8 committers/major contributors and Google just 1.
low	
Bugfix/Patches within 7 days	Proven history for minor releases (bugfixes) within 7 days. A few patches were released even in the same day.
high	
Release within 6 months	Track record that PostgreSQL major versions always was released for RDS in less than 6 months, sometimes even in less than 3 months.
high	
Near-Zero Downtime Upgrade	The default RDS option for MVU is based in pgupgrade without logical replication, therefore it requires downtime of dozens of minutes. However, the downtime can be reduced (usually under 60 seconds) with the RDS Blue/Green deployment for database updates.
	high / blocker
Escalation to RDS expert engineers	There's no specific support path to reach an Internal RDS engineer in a timely manner. However, RDS support has the SME (subject matter experts) program, which is part of Amazon Support escalation and should be available within minutes to collaborate with us on S1 incidents.
high	
Support custom Backup Plans	Amazon RDS supports custom backup plans and not only "daily" backups.

| Export of Backups with third-party tools | We regularly export base backups for testing, data analysis, export to Database Labs for the Database Team, or disaster recovery preparation, but third-party backup tools are not supported by RDS. The only option to export data for external usage is Snapshot Export into Apache Parquet format, which might not be supported by all consumers, or exporting pgdump to S3, which is slow to process | high / blocker |

| External Streaming Replication | RDS supports cross-region physical Replication, but it doesn't support physical replication to an external database.

| medium |

| WAL Archiving Delayed Replication | RDS PostgreSQL doesn't support recovery_min_apply_delay to configure delayed replication.

| medium |

| Debug capability | We lose observability (eg; strace/ perf) by not having full access to the database machine, however, if absolutely necessary RDS internal engineers might be escalated to perform any low-level debugging. Currently, we have evidences that this would happen in a timely manner.

| medium |

| No direct access to WAL files | We can't export WAL files nor execute pgwalddump into an RDS instance. This sometimes was required to debug frequently modified objects and their respective content

| low |

Things to validate

- We need to validate how to export RDS Performance Insights metrics into our Monitoring tools
- How to export PostgreSQL logs into Elastic
- Validate a connection pooling solution for both Write and Read-Only workloads
 - pgbouncer over VMs
 - RDS Proxy
- Compare Migration options
 - Native logical replication
 - Amazon Database Migration Services
- Evaluate time and impact of Major Version Upgrades 50k reference architecture?
 - Also evaluate Blue/Green deployments
- How long does it take to create a read-replica, or a new cluster from a backup? 10GB, 100GB, 1TB
- How long is the database service downtime with Blue/Green major version upgrade method?
 - We should also test if pgbouncer or RDS Proxy can hold requests to alleviate the impact during a Blue/Green deployment.

Evaluate changes over current Dedicated-RDS deployment

- Cost of Enable Enhanced Monitoring with small granularity (< 10 seconds)
- Evaluate offload of read operations into Standby Replicas
- Evaluate "Enable auto minor version upgrade"
- Evaluate performance gain with "Dedicated Log Volume"
- Increase logging level: slow queries, temp usage, autovacuum, lock waits, connections/disconnections, DDL statements
- Configure pgstatstatements settings
- Load and Configure autoexplain
- Implement "logical backup" solution

- Review Alarms: RDS Event Subscriptions, AWS Cloud Watch

Refactor Current Automation

The current platform to host all PostgreSQL clusters for GitLab.com is based mainly on Terraform and Chef, managing GCP virtual machines. Some complex operations are orchestrated via Ansible.

The main problems currently with this architecture is the historical growth and technical debt. It was designed to host a single, or small fixed number of database clusters. Hosting any arbitrary number of clusters is theoretically possible, but requires manual steps and overhead.

These inefficiencies are incompatible with the estimated requirements of hosting many database clusters, e.g. >100.

Some examples:

- For each database cluster we have to create multiple Chef roles
- For each database cluster we have to manually define a network subnet and write it in a configuration file
- By policy for each database cluster in production, we have an equivalent in staging

These shortcomings and bottlenecks could be removed by improving the automation with Chef or replacing it by a different tool set.

Pro	Description
Priority / Importance	
-----	-----
-----	-----
-----	-----
-----	-----
-----	-----
-----	-----
Iteration, Low Risk	We have an inefficient but fully working platform, therefore we can deploy enhancements iteratively, one step at a time. There is no "big bang" migration, no risk of not having certain features or to discover shortcomings of a new platform late in the process.
high	
Cons / Risks	Description
Priority / Importance	
-----	-----
-----	-----
-----	-----
-----	-----
-----	-----
-----	-----
Chef Know-How	Chef know-how and usage inside GitLab (and outside) is already decreasing. The current team does not have the needed know-how and capacity to implement the needed changes in a timely manner. Moving other SRE with Chef and domain knowledge into the DBRE team would be necessary, as well as GitLab's commitment to Chef.
high / blocker	
Efficiency	The ideal outcome will most likely still be not as efficient as a DBaaS solution.
medium	

| Development / Maintenance | We still have to develop and maintain the full automation code in-house.

| high / blocker |

| Integration | Each Cell will run in a single k8s cluster, this solution will not be Cell local. This brings higher complexity and possible failure vectors.

| medium |

Cells 1.0

Cells 1.0 is the first iteration for Cells, which should become GA during 2024.

This first iteration will only host a limited number of test users and projects and therefore does not have the full requirements.

It poses an opportunity to learn and iterate on the database platform.

During the development the number of Cells can grow from 1 to up to 10.

The estimate is that it will take multiple month until a second Cell will be created, and a second cluster is needed.

Cloud SQL currently does not fulfill the given high priority requirements, and there is no roadmap which fully includes them as well.

We will use Cloud SQL temporary for Cells 1.0 despite that to unblock the Cells 1.0 time-line. Therefor we acknowledge and agree to the associated risks and bottlenecks.

- There will be downtime to migrate to the final platform!
- The quality of service will not be equal to GitLab.com.
 - There is no clear SLO for deep level support.
 - No major version upgrades with reasonable downtime.
- We can not grow reliable grow this solution to the target of 50k users, we see problems for 3k users regarding connection pooling already.

We prioritize finding a solution meeting the requirements, to make it available after Cells 1.0.

The most promising candidates are the existing k8s PostgreSQL operators, offering PostgreSQL as a Service like experience.

Cells 1.x

As highlighted in Cells 1.0 we start with Cloud SQL as a temporary solution to free resource to build a sufficient platform for the long-term requirements.

The first phase will evaluate which k8s operators are available and which are the most fitting candidates for a full evaluation.

Current Database Platform

The current database platform is limited in future growth, but currently not at the edge of saturation.

It is not clear when the database will become a bottleneck, but based on the prior experience from before decomposition a database with 200% size poses operational challenges.

There are currently no predictions how long it takes for our database to double in size, but it is fair to assume this will not happen in the next 24 month, until 2026.

This gives enough time to migrate significant data into cells, or to improve / change the database platform here as well.

SECTION: engineering/architecture/design-documents/cells/infrastructure/runner.md

{{< engineering/design-document-header >}}

This blueprint describes the architecture and roadmap of the Runner service in a cellular architecture.

Introduction

GitLab Runner is a service that is relatively easy to provision and update compared to other GitLab.com services.

Currently having two exemplary setups, SaaS Runners and Dedicated Runners, with the latter having a superior setup enhanced by implementing GRIT.

Following Cell's philosophy, the Runner service is a perfect candidate for implementing a cell-local service, with almost no need for any change to its codebase(TBC).

In Cells 1.0, new customers are expected to enroll in the first Cell, in this iteration, we'll utilize the existing work made as part of Dedicated Beta release to provision the Runners.

Architecture

- There's a 1:n relationship between a Cell and a Runner, i.e, one Cell can have many Runners, while a Runner is only registered to one Cell.
- Legacy Cell (GitLab.com) is using Runners that are managed via config-mgmt and chef-repo, while the Cells are using Transistor-like setup, which vendors in GRIT.

Note, there's currently no Runner Reference Architectures, however, some links the reader can reference:

- [GitLab-runner/docs/fleetscaling](#).
- Create a test framework for testing runner fleet autoscaling configurations.
- Document jump start configurations for an autoscaled runner fleet on Google Compute.
- Runner Fleet Scaling and Configuration Best Practices Center.

Provision

Cells are expected to follow provisioning design and tooling similar to that of Dedicated; Runners in this case would utilize a tool similar to Transistor, which in a nutshell, is a

collection of Terraform modules and bash scripts.

Transistor's development guidelines have more elaborate info, but a simplified version of the provisioning steps of a Runner could look like the following:

1. Generate a runner model.
1. Provision the runner via one of the following steps:
 - Execute `./bin/prepare`, `./bin/onboard` then `./bin/provision`.
 - Alternatively, create an MR in Switchboard, which will execute the scripts above in order.

The deployment pipeline for a Runner could be triggered as a downstream pipeline following Instrumentor's deployment of a GitLab instance, this would require Instrumentor (or a higher-level job, e.g. in switchboard) to generate a runner model before the downstream pipeline is triggered.

Cells vs Dedicated

The Dedicated team has restricted access to the GitLab instances in which Runners are registered, this however, is not the case with Cells.

The SRE team fully owns and maintains a Cell, which means that registration token management will not require any external coordination, and can be a step that is automated using the provisioning scripts.

Deployment

Ideally, Cells:Runner should be managed similarly to the rest of the services in a Cell.

At this moment, GitLab.com SaaS Runners have a separate ad-hoc deployer tool, not to be confused with the GitLab.com grand Deployer, which is run and maintained by the Runner group.

In Cells, the Runner service will utilize Dedicated-like tooling, which is similar to the rest of the services, this will ensure visibility, observability and maintainability. Runners Dedicated are highly dependent on GRIT, which promises to introduce a blue-green mechanism which will be eventually utilized by both Dedicated and Cells.

Observability

Observability for Runners on Dedicated is currently a WIP, Runners should be treated as the rest of the services, and should utilize the recording rules and dashboards used currently in GitLab.com.

Roadmap

To keep Centralize tooling, 2 open questions:

1. How do we feel about eventually combining Transistor back into Instrumentor?

Combining Transistor and Instrumentor could "make changes to runners INCREDIBLY costly and slow

to have to go through the whole instrumentor deployment process." (@amknight , 2024), additionally, keeping both projects modular would enable a more efficient and safer release process as the QA testing process is separate (@skarbek, 2024).

1. How do we feel about incorporating Runner deployments with the rest of GitLab service deployments in a Cell?

As mentioned under Provision, we could possible use downstream pipelines to achieve this.

Iterations [WIP]

Cells 1.0

Linux Runners of type small will be available.

Cells 1.5

Cells 2.0

FAQ [WIP]

Q1

A1

Q2

A2

References

- @amknight (2024): !146768 (comment 1823581090).
- @skarbek (2024):!146768 (comment 1819418987).

SECTION: engineering/architecture/design-documents/cells/iterations/cells-1.0.md

This document describes a technical proposal for a Cells 1.0.

Cells 1.0 is a first iteration of the cellular architecture. The Cells 1.0 target is to deliver a solution for internal customers only, and be a foundational step for next iterations, to be able to deliver something with a smaller scope.

Cells 1.0 is a complete working feature that is meant to be deployed to GitLab.com SaaS.

Read more about Cells 1.5, which is meant to provide a mechanism to migrate existing customers and is built on top of the Cells 1.0 architecture.

Read more about Cells 2.0, which is meant to support the public and open source contribution model in a cellular architecture.

Preamble

A Cells 1.0 is meant to target internal customers that have the following expectations:

1. They want to use our multi-tenant SaaS solution (GitLab.com) to serve their Organization.
1. They accept that they may receive updates later than the rest of GitLab.com. Note that when GitLab does a patch release, the Delivery team makes sure that production (every cell) runs the patch release version before making the release public. This does not mean that outside of a patch release, all the cells run the same version of GitLab. See this private discussion for more information.
1. They want to use an environment with higher degree of isolation to rest of the system.
1. They want to control all users that contribute to their Organization.
1. Their groups and projects are meant to be private.
1. Their users don't need to interact with other Organizations, or contribute to public projects with their account.
1. Are OK with being unable to switch Organizations with their account.

From a development and infrastructure perspective we want to achieve the following goals:

1. All Cells are accessible under a single domain.
1. Cells are mostly independent with minimal data sharing. All stateful data is segregated, and minimal data sharing is needed initially. This includes any database and cloud storage buckets.
1. Cells need to be able to run independently with different versions.
1. A cluster-wide service is provided to synchronize state between all Cells.
1. A routing solution that is robust, but simple.
1. All identifiers (primary keys, user names, group, and project paths) are unique across the cluster, so that we can perform logical re-balancing at a later time. This includes all database tables, except ones using schemas gitlabinternal, or gitlabshared.
1. Because all users and groups are unique across the cluster, the same user can access other Organizations and groups at GitLab.com in Cells 2.0.
1. The overhead of managing and upgrading Cells is minimal and similar to managing a GitLab Dedicated instance. Cells should not be a linear increase in operational burden.
1. The Cell should be deployed using the same tooling as GitLab Dedicated.

This proposal is designed to cut as much scope as possible but it must not make it impossible to meet the following long-term goals:

1. Users can interact with many Organizations.
1. Cells can be re-balanced by moving Organizations between Cells.
1. The routing solution can dynamically route requests.

Proposal

The following statements describe a high-level proposal to achieve a Cells 1.0:

1. Terms used:

- 1. Cell: A single isolated deployment of GitLab that connects to the Topology Service.
- 1. Topology Service: The central service that is the authoritative entity in a cluster.
Provides uniqueness and routing information.

1. Organization properties:

- 1. We allow users to create a new Organization on a Cell.
The chosen Cell would be controlled by GitLab Administrators.
- 1. The Organization is private, and cannot be made public.
- 1. Groups and projects can be made private, but not public.

1. User properties:

- 1. Users are created on the Cell that contains the Organization.
- 1. Users are presented with the Organization navigation, but can only be part of a single Organization.
- 1. Users cannot join or interact with other Organizations.
- 1. User accounts cannot be migrated between Cells.
- 1. A user's personal namespace is created in that Organization.

The following statements describe a low-level development proposal to achieve the above goals:

1. Application properties:

- 1. Each secret token (personal access token, build token, runner token, etc.) generated by the application includes a unique identifier indicating the Cell, for example us0. The identifier should try to obfuscate information about the Cell.
- 1. The session cookie sent to the client is prefixed with a unique identifier indicating the Cell, for example us0.
- 1. The application configuration includes a Cell secret prefix, and the location of the Topology Service.
- 1. User always logs into the Cell on which the user was created.

1. Database properties:

- 1. Each primary key in the database is unique across the cluster. We use database sequences that are allocated by the Topology Service.
- 1. We require each table to be classified: to be cluster-wide or Cell-local.
- 1. We follow a model of eventual consistency:
 - 1. All cluster-wide tables are stored in a Cell-local database.
 - 1. All cluster-wide tables retain unique constraints across the whole cluster.
 - 1. Locally stored cluster-wide tables contain information required by this Cell only.
 - 1. The cluster-wide tables are restricted to be modified by the Cell that is authoritative over the particular record:
 - 1. The user record can be modified by the given Cell only if that Cell is the authoritative source of this record.
 - 1. In Cells 1.0 we are likely to not be replicating data across cluster, so the authoritative source is the Cell that contains the record.
 - 1. The Topology Service serves as a single source of truth for the uniqueness constraint (be it ID or user, group, project uniqueness).

1. All Cells use gRPC to claim usernames, groups or projects.
1. The Topology Service holds metadata information that allows to know on which Cell the username, group or project is.
1. The Topology Service does not hold source information (actual user or project records), only references that are indicative of the Cell where that information is stored.

1. Routing properties:

1. We implement a routing service that performs secret-based routing based on the prefix.
1. The routing service is implemented as a Cloudflare Worker and is run on edge. The routing service defines a set of rules, and uses Topology Service to classify how to route data.
1. Cells are exposed over the public internet, but might be guarded with Zero Trust.

Architecture overview

plantuml
@startuml

```
cloud "Cloudflare" as CF {
  [Routing Service Worker] as CFRSW
}
```

```
node "GitLab Inc. Infrastructure" {
  node "Cell Services" as CellServices {
    [Topology Service] as TS
    [Cloud Spanner] as TSCS
  }
}
```

```
TS --> TSCS
}
```

```
node "A Cell" as Cell {
  frame "GitLab Rails" as CellRails {
    [Puma + Workhorse + LB] as CellPuma
    [Sidekiq] as CellSidekiq
  }
}
```

```
[Container Registry] as CellRegistry
```

```
database DB as CellDB {
  frame "PostgreSQL Cluster" as CellPSQL {
    package "ci" as CellPSQLci {
      [gitlabci] as CellPSQLgitlabci
    }
  }
}
```

```
package "main" as CellPSQLmain {
  [gitlabmainclusterwide] as CellPSQLgitlabmainclusterwide
  [gitlabmaincell] as CellPSQLgitlabmaincell
}
```

```

    PCPSQLmain -[hidden]-> CellPSQLci
  }

  frame "Redis Cluster" as CellRedis {
    [Redis (many)] as CellRedismany
  }

  frame "Gitaly Cluster" as CellGitaly {
    [Gitaly Nodes (many)] as CellGitalymany
  }
}

CellRails -[hidden]-> CellDB
}

}

CFRSW --> CellPuma
CFRSW --> CellRegistry
CFRSW --> CellPuma
CFRSW --> CellRegistry
CFRSW --> TS
CellPuma --> TS
CellSidekiq --> TS

@enduml

```

Problems

The following technical problems have to be addressed:

1. All secrets are prefixed as this is required by a simple routing layer to perform secret-based routing.
1. All usernames, Organizations, top-level groups (and as result groups and projects) are unique across the cluster.
1. All primary key identifiers are unique across the cluster.

GitLab Configuration

The GitLab configuration in gitlab.yml is extended with the following parameters to:

```

yaml
production:
  gitlab:
    topologyservice:
      address: https://cell1.gitlab.com
      certificate: ...
      secretsprefix: kPptz

```

Topology Service

All services supported are described in dedicated documented about Topology Service.

Pros

- The proposal is lean:
 - Each Cell holds only a fraction of the data that is required for the cluster to operate.
 - The tables marked as mainclusterwide that are stored locally can be selectively replicated across Cells following a mesh-architecture.
 - Based on the assumption that Cells require only a fraction of shared data (like users), it is expected that Cells might need a small percentage of records across the whole cluster.
- The Topology Service is a single point of failure:
 - Reduced set of features allows to make it highly-available service.
 - Use highly-available database solution (Cloud Spanner).
- The routing layer makes this service very simple, because it is secret-based and uses prefix.
 - Reliability of the service is not dependent on Cell availability. It depends on availability of Topology Service to perform classification.
- Mixed-deployment compatible by design.
 - We do not share database connections. We expose APIs to interact with cluster-wide data.
 - The application is responsible to support API compatibility across versions, allowing us to easily support many versions of the application running from day zero.
- Database migrations.
 - Work out of the box, and are isolated to the Cell.
 - Because we don't share databases across Cells, the Cell has full ownership of mainclusterwide tables.
 - Fixing all the cross-joins between cluster-wide tables and cell-local tables is a large work.

Storing mainclusterwide tables locally in each cell allows us to save time for the first iteration.

Cons

- We intentionally split data for tables marked as mainclusterwide across all cluster cells.
 - These tables will be selectively replicated (likely) outside of the application to allow Cells to share data.
 - We effectively reinvent database replication to some extent.
- We are severely limited by how many tables can be made mainclusterwide. Exposing many tables is a significant amount of additional code to allow cross-Cell interaction.
 - We require all tables to be classified. We want to ensure data consistency across the cluster if records are replicated.

Features on GitLab.com that are not supported on Cells

For the initial deployment of Cells 1.0, we are cutting scope on some features to get something deployed.

This doesn't mean that Cells 1.0 is not going to support these in the future, but our application/infrastructure doesn't support them yet.

The table below is a comparison between the existing GitLab.com features, and not self-managed/Dedicated.

No Initial Support	Reasoning

GitLab Pages	Complexity.
CI Catalog	CI Catalog depends on public projects, organizations in Cells 1.0 can't see public projects.
Organization Switching	A user belongs to a single organization.
Shared user accounts across Cells	Users will need to have new user accounts on each Cell for now
GitLab Duo Pro license works across all projects on instance GitLab Duo Pro licenses, once granted, should allow users to use GitLab Duo Pro on all projects on the instance. With Cells 1.0, this will only work within their own cell.	
User removal	Users can only be part of one Organization. A removal would equal a deletion in this case, so only user deletions will be offered in Organizations on Cells 1.0. Upon removal, there would be no way for a User to discover another Organization to join, as they are private for Cells 1.0.
Hosted runners on Windows and macOS Hosted runners on Windows and macOS runners are still in beta and there are some more complex technical considerations related to cost. See the discussion: 434982 (comment 1789275416) on sharing resources. Self-managed runners are supported.	
Multiple Sizes for Linux Runners	We will only support small linux runners on Cells 1.0.
GitLab for Jira Cloud app and GitLab for Slack app integrations Jira and Slack apps can only be configured to post to single endpoints, so there is nothing in the configured endpoints' routes that would allow the Cells router to know which cell to route to. We may need to support at the organization level. See 467791 and 467809 for more details.	
Cross-organization downstream pipelines Private organizations are in Cells 1.0 only and downstream pipelines would be unable to see public organizations.	
Any feature dependent on Clickhouse Clickhouse is not supported on Dedicated, which is the underlying provisioning tool for Cells. Clickhouse is also not supported in any of our other tooling such as Geo, Org Mover, Backup/Restore, etc.	
Any feature dependent on incoming email (mailroom) Cut scope. While we have a proposal to have ingest email per cell, we are yet to figure out how to have stable email addresses that can be used even when an organizations moves to a different cell.	

| Global search | Each cell will have an isolated search cluster. With Cells 1.0, global search will only work within the cell. See the Cells: Global Search design document for more details.

|

| Paid subscription flows | CustomersDot relies on path based and OAuth token routing for Single Sign-On and fetching/updating data on GitLab. Without these, all requests from CustomersDot will go to the legacy Cell. |

| Legacy CIJOBToken | The legacy CIJOBToken cannot be routed because it does not contain routing information, and can be passed in the request body. Customers will need use the JWT format in order to be able to utilise CIJOBToken outside of the legacy cell. |

| OAuth Provider / OIDC Provider | OAuth/OIDC application provide a mechanism for third party and first party services to integrate to Gitlab. Since cross cell communication is limited in cells 1.0, we can not reliably implement OAuth applications |

| Group SAML | Group SAML will not be available outside legacy cell open issue for cell 1.5|

Phases

All Cells 1.0 work is tracked under the Cells 1.0 Epic.

The Epic is split into multiple phases where each one represents a iteration to achieve Cells 1.0.

Some of these phases have dependencies over one another, and some can be run in parallel.

Phase 1: PreQA Cell

Exit Criteria:

- New GCP organizations created.
- Break glass procedure.
- Ring definition exists.
- Cell provisioned using dedicated stack.
- Able to do configuration changes to Cell.
- Cell available at xxx.cells.gitlab.com.
- Cell doesn't handle data uniqueness.

source

Unblocks:

- Phase 3: To provision runway deployment for Topology Service
- Delivery team: Start testing deploys on rings

Dependencies:

- None

Details:

- Epic
- DAG

Phase 2: GitLab.com HTTPS Passthrough Proxy

Exit Criteria:

- 100% of API traffic goes through router using passthrough proxy rule.
- 100% of Web traffic goes through router using passthrough proxy rule.
- 100% of Git HTTPS traffic goes through router using passthrough proxy rule.
- Requests meet latency target
- registry.gitlab.com not proxied.

source

Unblocks:

- Phase 3: Router to be configured with additional rules in phase 3.

Dependencies:

- None

Details:

- Epic
- DAG

Phase 3: GitLab.com HTTPS Session Routing

Exit Criteria:

- PreQA Cell configured to generate gitlabsession with prefix using rails config.
- Route gitlabsession with matching prefix to PreQA Cell using TopologyService::Classify (REST only) with static config file.
- Continuous Delivery on Ring 0 with no rollback capabilities and doesn't block production deployments.
- Topology Service Readiness Review for Experiment
- Topology Service gRPC endpoint not implemented.

Unblocks:

- Phase 4

Before/After:

source

Dependencies:

- Phase 2: Passthrough proxy needs to be deployed.
- Phase 1: GCP organizations, Ring definition exists.

Details:

- Epic
- DAG

Phase 4: GitLab.com HTTPS Token Routing

Exit Criteria:

- Framework to generate routable tokens in Rails.
- Framework to classify routable tokens in HTTP Router.
- Topology Service being able to classify based on more criteria.
- Route Personal Access Tokens to different Cells using TopologyService::Classify.
- Support PRIVATE-TOKEN: and Authorization: HTTP headers for Personal Access Tokens, create issues for other to be solved in following phases.
- Each routing rule added should be covered with relevant e2e tests.
- Route Job Tokens and Runner Registration to different Cells using TopologyService::Classify.

Dependencies:

- Phase 3: Topology Service and Router need to running in production.

Before/After:

source

Details:

- Epic
- DAG

Phase 5: Cluster Awareness

Exit Criteria:

- Topology Service Production Readiness Review for Beta.
- Framework to claim resources globally using TopologyService::Claims storing them in Google Spanner.
- Following resources are claimable; Username, E-Mail, Top level Group Name, Routes
- All resources that need to be claimed identified.
- Unique indexes are audited, to not break any uniqueness required by application, and allows data migration.
- Lease a sequence to a Cell using TopologyService::Sequence.
- Rails application able to send requests to TopologyService using internal network.
- mTLS communication between TopologyService and HTTP Router.

- mTLS communication between TopologyService and Rails.
- mTLS communication between HTTP Router and Cell.
- PreQA Cell can start claiming resources, still detached from Legacy Cell.
- Claims done by PreQA Cell will be deleted.

Dependencies:

- Phase 3: Topology Service Deployed.

Before/After:

source

Details:

- Epic
- DAG

Phase 6: Monolith Cell

Exit Criteria:

- Topology Service Production Readiness GA.
- Legacy Cell configured as a Cell in TopologyService.
- All new resources in Legacy Cell are claimed using TopologyService::Claims.
- Legacy Cell claimed all existing resources.
- Sequence leased to Legacy Cell.
- Capacity Planning for sequences leased.
- Latency increase for creating globally unique resources up to 20ms.

Dependencies:

- Phase 5: Cluster Awareness

Before/After:

source

Details:

- Epic
- DAG

Phase 7: Cell Initialization

Exit Criteria:

- TBD

Before/After:

Details:

- Epic
- DAG

Phase 8: Organization Onboarding

Exit Criteria:

- TBD

Before/After:

Details:

- Epic
- DAG

Phase 10: Production Readiness

Exit Criteria:

- Cell-Level Observability (Logs, Metrics, Alerts, Dashboard).
- Integration with existing Incident Management tooling.
- Compliance with GitLab.com security standards.
- Regional and Zonal Disaster Recovery capabilities.
- Operational tooling independence from GitLab.com/dev.gitlab.org availability.
- Centralized WAF management for GitLab.com domain.
- Cell-level Application Rate Limits with synchronization.
- Least-privileged access implementation with SRE escalation path.
- Progressive rollout of infrastructure changes across Cells with rollback support.
- Progressive deployment capabilities across Legacy Cell and Cells with rollback support.
- Support for toggling Feature Flags across Legacy Cell and Cells.

Dependencies:

- Phase 1: GCP organizations, Ring definition exists.

Before/After:

Details:

- Epic

Questions

1. How will we onboard users to an Organization on additional Cells?

An Admin will perform the following tasks:

1. Create an Organization on the additional cell.
1. Create a new user with the Owner role in the Organization.
1. Remove the Admin from the Organization. Optional, depending on feature set.
1. The new Owner will import data for this group. This would create users, add them to the groups/projects, and add them to the Organization.

1. How do we register new users for the existing Organization on an additional Cell?

The standard group and project invite flows can be used. This means a user with adequate permissions can invite users by email to any group or project in the Organization. After the user registers they will be added to the group or project and the Organization.

1. How would users log in?

- UI: The login to Organizations would be scoped to the Organization:
<https://<GITLABDOMAIN>/users/signin?organization=gitlab-inc>.
- SAML: <https://<GITLABDOMAIN>/users/auth/saml/callback> would receive ?organization=gitlab-inc which would be routed to the correct Cell.
- This would require using the dynamic routing method with a list of Organizations available using a solution with high availability.

1. How do we add a new table if it is initially deployed on additional Cell?

The Topology Service is ensuring uniqueness of sequences, so it needs to have sequence.

1. Is Container Registry cluster-wide or cell-local?

The Container Registry can be run Cell-local, and if we follow the secret-based routing, it can follow the same model for filtering.

We would have to ensure that the JWT token signed by GitLab is in a form that can be statically routed by the routing layer.

1. Are GitLab Pages cluster-wide or Cell-local?

GitLab Pages was determined to be non-essential for Cells 1.0, so we would not support them for Cells 1.0.

The discussion about this can be found [here](#).

If GitLab Pages are meant to support the .gitlab.io domain:

- GitLab Pages need to be run as a single service that is not run as part of a Cell.
- Because GitLab Pages use the API we need to make them routable.
- Similar to routes, claim pagesdomain on the Topology Service
- Implement dynamic classification in the routing service, based on a classification key.
- Cons: This adds another table that has to be kept unique cluster-wide.

Alternatively:

- Run GitLab Pages in a Cell, but provide a separate domain.
- Custom domains would use the separate domain.
- Cons: This creates a problem with having to manage a domain per Cell.
- Cons: We expose Cells to users.

1. Should we use a static secret for the internal endpoint or JWT?

To be defined. The static secret is used for simplicity of the presented solution.

1. How do we handle SSH cloning?

This is a separate problem tangential to routing, which is intentionally not solved by this proposal.

The problems associated with SSH cloning are:

- We need to validate a user public key to identify the Cell holding it.
- We need to ensure uniqueness of public keys across the cluster.

1. Are there other cluster-wide unique constraints?

- Authorized keys
- GPG keys
- Custom e-mails
- Pages domains
- TBD

1. How do we synchronize cluster-wide tables, such as broadcast messages or application settings?

We would very likely take a naive approach: expose those information using API, and synchronize it periodically.

Tables likely to be made synchronized are:

- Application Settings
- Broadcast Messages
- TBD

1. How do we dogfood this work?

To be defined.

1. How do we manage admin accounts?

Since we don't yet synchronize across the cluster, admin accounts would have to be provided per Cell.

This might be solved by GitLab Dedicated already?

1. How are secrets are generated?

The Cell prefix is used to generate a secret in a way that encodes the prefix. The prefix is added to the generated secret.

Example:

- GitLab Runner Tokens are generated in the form: glrt-2CR8XYZ
- For Cell prefix: secretsprefix: kPptz
- We would generate Runner tokens in the form: glrt-kPptz2CR8XYZ

1. Why secrets-based routing instead of path-based routing?

Cells 1.0 are meant to provide a first approximation of the architecture allowing to deploy a multi-tenant and multi-Cell solution quickly.

Solving path-based routing requires significant effort:

- Prefixing all existing routes with `/org/<org-name>/-/` will require a deliberate choice how to handle ambiguous or Cell-local or cluster-wide routes, such as `/api/v4/jobs/request`. Changing routes also introduces breaking changes for all users that we would migrate.
- Making all existing routes routable would be a significant effort to fix for routes like `/-/autocomplete/users` and likely a multi-year effort. Some preliminary analysis how many routes are already classified can be found in this comment.
- In each case the routing service needs to be able to dynamically classify existing routes based on some defined criteria, requiring significant development effort, and increasing the dependency on the Topology Service.

By following secret-based routing we can cut a lot of initial complexity, which allows us to make the best decision at a later point:

- The injected prefix is a temporary measure to have a static routing mechanism.
- We will have to implement dynamic classification anyway at a later point.
- We can figure out what is the best way to approach path-based routing after Cells 1.0.

1. What about data migration between Cells?

Cells 1.0 is targeted towards internal customers. Migrating existing customers, and onboarding new customers is a big undertaking on its own:

- Customers to be migrated need to opt into the Organization model first.
- New Customers are not willing to have missing features
- We expect most of the isolation features to work.
- We have to transparently move data from the source Cell to the target Cell. Initially we would follow a model of Cell split. We would clone the Cell and mark where the given record is located.

Existing customers could still be migrated onto a Cells 1.0, but it would require to use import/export

features similar to how we migrate customers from GitLab.com to Dedicated. As a result, not all information

would be migrated. For example, the current project import/export does neither migrate CI traces, nor job artefacts.

1. Is secret-based routing a problem?

To be determined.

1. How would we synchronize users across Cells?

We build out-of-band replication of tables marked as mainclusterwide. We have yet to define if this would be better to do with an API that is part of Rails, or using the Dedicated service.

However, using Rails would likely be the simplest and most reliable solution, because the application knows the expected data structure.

Following the above proposal we would expose users and likely all adjacent tables using API:

/api/v4/internal/cells/users/:id. The API would serialize the users table into a protobuf data model.

This information would be fetched by another Cell that would synchronize user entries.

1. How would the Cell find users or projects?

The Cell would use Classify Service of Topology Service.

1. Would the User Profile be public if created for enterprise customer?

No. Users created on another Cell in a given Organization would be limited to this Organization only.

The User Profile would be available as long as you are logged in.

1. What is the resiliency of a Topology Service exposing cluster-wide API?

The API would be responsible for ensuring uniqueness of: User, Groups, Projects, Organizations, SSH keys, Pages domains, e-mails.

The API would also be responsible for classifying a classification key for the routing service.

We need to ensure that the Topology Service cluster-wide API is highly available.

1. How can instance-wide CI runners be configured on the new cells?

To be defined.

1. Why not use FQDN (mycorp.gitlab.com or mygitlab.mycorp.com) instead?

We want the gitlab.com to feel like a single application, regardless how many organizations you interact with.

This means that we want to share users, and possibly some data between organizations at

later point.

Following model of mycorp.gitlab.com or mygitlab.mycorp.com creates a feel of a strong isolation between instances,

and different perception of how system operates compared to how we the system to actually behaves.

1. Are feature-flags cell-local or cluster-wide?

To be defined.

1. How the Cells 1.0 differ to GitLab Dedicated?

- The GitLab Dedicated is a single-tenant hosted solution provided by GitLab Inc. to serve a single customer under a custom domain.
- The Cellular architecture is meant to be multi-tenant solution using gitlab.com domain.
- The GitLab Dedicated by design has all resources separate between tenants.
- The Cellular architecture does share resources between tenants to achieve greater operational efficiency and lower cost.

1. Why the Organization is private, and cannot be made public in Cells 1.0?

- Private organizations on Cells do not require any data sharing or isolation as this is achieved by the current system.
- We don't have to do extensive work to isolate organizations yet.
- Routing is simpler since we anticipate all requests to be authenticated, making them easier to route.
- We don't have to "fix" routes to make them sharded yet.

1. Why not to prefix all endpoints with the relativepath for all organizations?

This breaks the main contract of what we want to achieve:

- Migration to use organizations is seamless.
- We do not want to break existing user workflows if user migrates their groups to organization,
or when we migrate the organization to another Cell.
- For the following reason this is why we don't want to force particular paths, or use of subdomains.
 - If we choose the path to force to use relativepath it would break all cell-wide endpoints
This seems to be longer and more complex that approaching this by making existing to be shared.
 - If we choose to fix existing not sharded can be made
at later point we will achieve much better API consistency, and likely much less amount of the work.

1. Why not use subdomains, like mycorp.gitlab.com?

Reasons to avoid using DNS for routing:

- Risks setting an expectation of full isolation, which is not the case.
- Care must be taken to prevent security issues with cookies leaking across subdomains.
- Complicates working across Organizations, with changing hostnames and a requirement to pass full hostname in all links.
- Complicates integrations and API interactions, as the URL and host will change based on the organization.
- We will need to build a common login service, which will redirect users to a valid organization/URL.
- Increased risk of naming collisions with higher impact. For example different divisions or organizations across a large enterprise, or companies with similar names.

1. What about Geo support?

The Geo support is out of scope of Cells 1.0. However, it is fair to assume the following:

- Geo is per-Cell.
- Routing Service can direct to Geo replica of the Cell (if it knows it).
- We might have many routing services in many regions.

1. Are cluster-wide tables available to all cells?

No, cluster-wide tables are stored in a Cell-local database. However, we will determine synchronization of cluster-wide tables on a case by case basis.

1. How can I adapt a feature to be compatible with Cells?

Many groups have questions about how to adapt a feature for Cell. This especially applies if a feature is available at the instance level, or can be used across groups.

Here are some strategies for evolving a thing for Cells 1.0:

- Leave the feature unchanged.
For example, admins / users will have to create an account per cell.
- Disable the feature for Cells 1.0.
- For critical cases, move the feature to cluster-wide level.
For example, users can sign in at a single location, <https://gitlab.com/users/signin>.

In many cases, it is not yet necessary to re-implement an instance-wide feature to work on a cluster-wide level. This is because for Cells 1.0, the net effect of only allowing private visibility and new users mean that we can defer this until Cells 1.5.

SECTION: engineering/architecture/design-documents/cells/iterations/cells-1.5.md

This document describes a technical proposal for a Cells 1.5 that builds on top of Cells 1.0.

The Cells 1.5 target is to deliver a migration solution for existing enterprise customers using the SaaS GitLab.com offering.

Preamble

Cells 1.5 is meant to target existing enterprise customers:

1. They are existing customers of the GitLab.com Legacy Cell and want to use the Organization model.
1. They want their Organization to be isolated from the rest of GitLab.com.
1. Their groups and projects that are meant to be private.

From a development and infrastructure perspective we want to achieve the following goals:

1. Customers that migrated to the Organization model are isolated from each other.
1. Users can be members of multiple Organizations.
1. We can migrate Organizations from the Legacy Cell to another Cell without user intervention or changing any user workflows.
1. The routing solution can dynamically route customers to the correct Cell once they are migrated.

Long-term we want to achieve the following goals:

1. The proposed architecture must not prevent us from implementing the ability for users to interact with many Organizations.
1. Cells can be re-balanced by moving Organizations between Cells.

SECTION: engineering/architecture/design-documents/cells/iterations/cells-2.0.md

This document describes a technical proposal for a Cells 2.0 that builds on top of Cells 1.5.

The Cells 2.0 target is to support a public and open source contribution model in a cellular architecture.

Preamble

Cells 2.0 is meant to target public and open source Organizations on GitLab.com:

1. Existing users can create public Organizations that are isolated from the rest of GitLab.com.
1. A single user can be part of many Organizations that are on different Cells.
1. Users can contribute to public projects across Cells.

From a development and infrastructure perspective we want to achieve the following goals:

1. We can migrate public Organizations between Cells without user intervention or a user changing any of their workflows.
1. The routing solution allows seamless interaction with many Organizations at the same time.

SECTION: engineering/architecture/design-documents/cells/rejected/deployment-architecture.md

{{< engineering/design-document-header >}}

This blueprint was surpassed by the infrastructure blueprint

This section describes the existing deployment architecture of GitLab.com and contrasts it with the expected Cells architecture.

1. Before Cells - Monolithic architecture

The diagram represents simplified GitLab.com deployment components before the introduction of a Cells architecture.

This diagram intentionally omits some services that are not relevant for the architecture overview (Cloudflare, Consul, PgBouncers, ...).

Those services are considered to be Cell-local, with the exception of Cloudflare.

The component blocks are:

- Separate components that can be deployed independently.
- Components that are independent from other components and offer a wide range of version compatibility.

The application layer services:

- Are strongly interconnected and require to run the same version of the application.
Read more in !131657.
- Each service is run across many nodes and scaled horizontally to provide sufficient throughput.
- Services that interact with other services using an API (REST, gRPC), Redis or DB.

The dependent services:

- Are updated infrequently and selectively.
- Might use cloud managed services.
- Each service is clustered and might be run across different availability zones to provide high availability.
- Object storage is also accessible directly to users if a pre-signed URL is provided.

2. Development Cells - Adapting application to Cellular architecture

The purpose of Development Cells is to model a production-like architecture to test and validate the changes introduced.

This could be achieved with testing Cells on top of the Reference Architectures.

Read more in 425197.

The differences compared to Before Cells are:

- A Routing Service is developed by Cells.
- Development Cells are meant to be run using a development environment only to allow prototyping of Cells without the overhead of managing all auxiliary services.
- Development Cells represent a simplified GitLab.com architecture by focusing only on essential services required to be split.
- Development Cells are not meant to be used in production.
- Cluster-wide data sharing is done with a read-write connection to the main database of Cell 1: PostgreSQL main database, and Redis user-sessions d